ANLP FInal Report

Salman Tariq

For this project, I worked on fine-tuning a T5 model to simplify texts according to CEFR levels. The model is designed to take a complex sentence and generate a simpler version at the target level, which could range from A1 to C2. During the development, I prepared the dataset by cleaning it and removing obvious anomalies, such as sentences with missing or incorrect labels. To make the model more robust, I applied data augmentation techniques. This included paraphrasing sentences, swapping synonyms, and slightly modifying sentence structures while keeping the original meaning intact. The goal was to expose the model to more varied examples, helping it generalize better to unseen data.

I trained the model for 5 epochs. While this gave me initial outputs for the test set, the simplifications were not as accurate or refined as expected. At that stage, I generated simplified texts and created a submission file, but I did not run the official evaluation script to compute metrics like weighted F1, adjusted accuracy, or BERT-based scores.

Adjustments: During the final presentation, after observing the work of other teams and group members, I realized that my workflow was incomplete because I had not executed the evaluation script to measure the model performance. I then ran the evaluation using the official script, confirmed that all test instances were covered. These adjustments ensured that the project included the full pipeline: from data cleaning and augmentation, through model training, to generating outputs and evaluating them properly.

The Markdown file includes the group members and the corresponding filenames for clarity and reference.

Dongxin Wang

While peer approaches primarily focused on data-centric techniques such as data augmentation (paraphrasing/synonym swapping) on a single architecture (T5), my work centered on architectural comparison and inference-time optimization. I implemented and fine-tuned two distinct Seq2Seq models: Google's FLAN-T5 and Facebook's BART and evaluated their respective capabilities in control-based text simplification. Furthermore, I addressed the common "conservative generation" issue (where models merely copy the input) by engineering specific inference hyperparameters, rather than relying solely on training data modifications.

Here are some key improvements:

- **Dual-Model Architecture.** Unlike the baseline approach of using only T5, I extended the experiment to include BART. T5 treats simplification as a "text-to-text" transfer task,

while BART is pre-trained as a denoising autoencoder. I hypothesized that BART might produce more natural rewriting and structural simplification. And I built a modular pipeline capable of switching between architectures, utilizing *AutoModelForSeq2SeqLM* for T5 and *BartForConditionalGeneration* for BART, sharing the same training loop for fair comparison.

- **Control Token Injection** (Light Prompt Engineering). To explicitly guide the models towards specific CEFR levels (A2/B1), I implemented a prompting strategy during data preprocessing. Instead of feeding raw text, I injected control tokens into the source sequence: *Simplify to [Level]: [Original Text]*. This enabled a single model to handle multi-level simplification dynamically based on the input instruction, ensuring alignment with the TSAR-2025 task requirements.
- **Performance Optimization**. A major challenge observed was the model's tendency to reproduce the source text (high BERTScore-Orig but low Simplification score). Initially, with the default parameters of Repetition Penalty (1.0), I observed that the model was simply copying the original text. So I tried to understand the parameters increasing the repetition penalty to 1.5, but the fine tuning became extremely slow. So, I gradually lowered it and found that 1.2 was the 'sweet spot'. Same for the other parameters, I implemented generation constraints during the inference phase:
    1. Length Penalty (1- > 0.8): Encouraged the generation of shorter sequences to structurally simplify complex sentences.
    2. Beam Search (k=5): Increased beam width to explore more diverse simplification paths compared to standard greedy decoding.

For the evaluation, I executed the official `evaluation script` to compare both models against the ASSET validation set. My results demonstrated that BART outperformed T5 across key metrics, achieving a higher Weighted F1 (0.35 vs. 0.31) and better Adjusted Accuracy (0.76 vs. 0.73). And while both models achieved high semantic preservation scores (~0.99 BERTScore-Orig), the inference adjustments successfully pushed the models towards valid simplification.

TzuLun Yeh

For my contribution, I focused on a Prompt Engineering Ablation Study using the Google Gemini-2.5-flash model. I designed four distinct strategies to evaluate how different context types affect simplification quality: a Zero-shot baseline, Instructions-only (applying specific linguistic constraints), Few-shot examples (utilizing in-context learning), and a Merged strategy combining both.

To handle the large scale of API requests efficiently, I engineered a high-performance execution pipeline. I utilized Python's `ThreadPoolExecutor` to enable 20 concurrent workers, drastically reducing the inference time. I also implemented a robust fault-tolerance system, including exponential backoff for API rate limits and a JSONL checkpointing mechanism, ensuring that data was saved atomically to prevent loss during long runs.

For evaluation, I built an automated pipeline integrating an ensemble of ModernBERT-based CEFR classifiers, MeaningBERT, and BERTScore. The results were significant: while the Baseline strategy frequently over-simplified B1 texts (achieving only 25% accuracy), the Merged strategy achieved the highest performance with 69% accuracy on B1 targets. This notably outperformed the classifier's benchmark for human-written references (65%), proving that combining linguistic constraints with few-shot examples effectively mitigates over-simplification and achieves precise CEFR targeting.